6.28 指令流水线——数据通路

考纲定位：计算机组成原理 · 第 6 章 中央处理器（6.28） | 重要度 ★★★★ | 常见考法：选择题/大题（五段流水线、流水寄存器、控制信号下传）

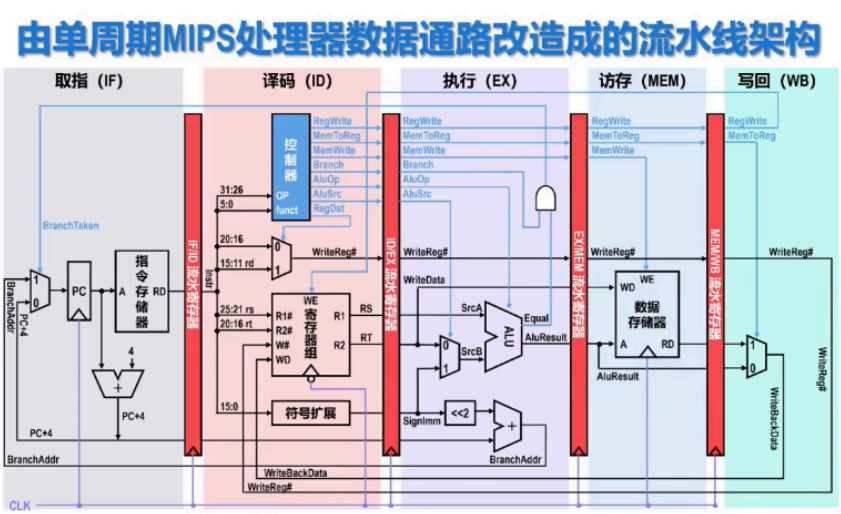
一句话核心：流水线数据通路 = 五段专用通路 + 段间寄存器；控制信号随指令一起下传

图旁说明：图只保留原图；文字补充考点与易错，不重复图中五段流程

考点/易错

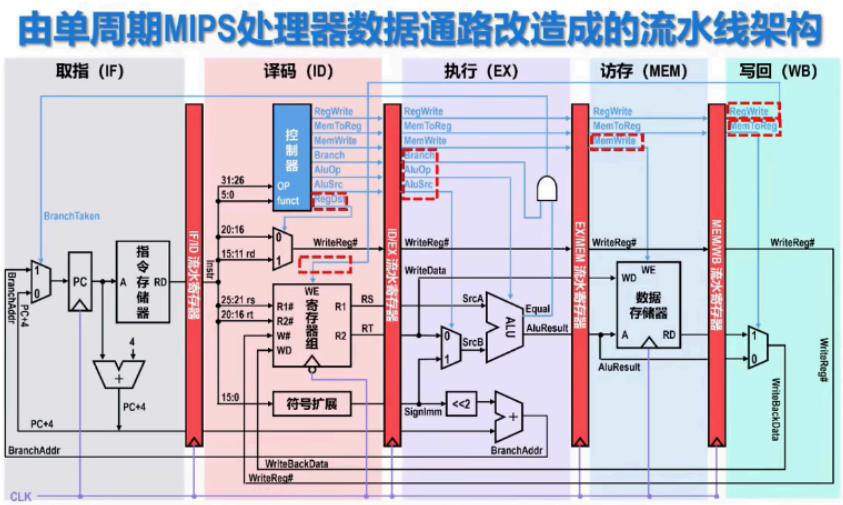
- 流水寄存器既传数据又传控制信号；后级需要该指令的 RegWrite/MemRead/ALUSrc 等
- 写回结果要回头送寄存器堆，分支结果要回头改 PC；这两条“回头路”是冒险的根源
- 寄存器堆通常“前半周期写、后半周期读”，避免 ID 读与 WB 写冲突
- load-use 即使有旁路也要停 1 拍，因为 MEM 段末才出数据

一、结构



- 我们之前并未给同学们介绍过**单周期数据通路**（一种专用通路结构的数据通路），而介绍的是**基于CPU内部单总线结构的数据通路**。
- 对于采用专用通路结构的CPU，其内部**各寄存器之间或寄存器与ALU之间采用的都是专用数据传输通路连接**，而不是单总线结构的共享连接。因此，**可在各专用通路中并行传输数据**，控制起来也比单总线结构CPU简单。

关于流水寄存器存储的内容

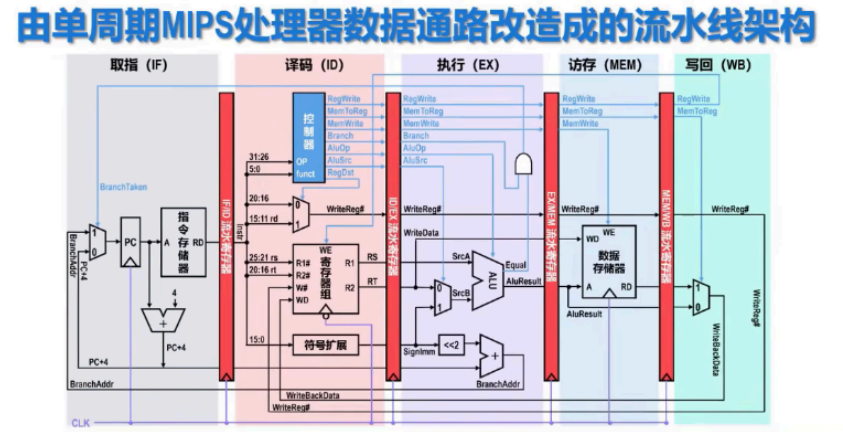


- 与单周期处理器使用相同的操作控制信号，复用单周期处理器中的操作控制器。
- 在指令流水线中，**通过流水寄存器传递以下内容**：
  - **当前指令。**
  - **当前阶段待加工的数据。**
  - **后序功能段数据加工所需的操作控制信号。**

✓ ID功能段负责指令译码生成操作控制信号，因此操作控制器应该设置在ID功能段中。请注意：操作控制器用于控制数据通路，而不属于数据通路。

✓ 所有控制信号由操作控制器一次产生，逐功能段使用。每个操作控制信号只在某一功能段内使用一次，使用完毕后就不再向后段继续传递。

关于若干控制信号



控制信号	功能段	来自	作　用
BranchTaken	IF	EX	分支跳转控制，1表示跳转，由EX功能段的Branch信号与equal标志位进行逻辑与运算得出
RegDst	ID	ID	目的寄存器选择，1表示选择rd寄存器，0表示选择rt寄存器
RegWrite	ID	WB	寄存器组写操作控制，1表示数据需要写回寄存器组中的指定寄存器
AluSrc	EX	EX	ALU的第二输入选择，1表示输入扩展后的立即数SignImm，0表示输入rt寄存器
AluOp	EX	EX	ALU运算控制，用于控制ALU进行不同运算，具体取值与位宽与ALU的设计相关
MemWrite	MEM	MEM	数据存储器写操作控制，1表示进行写操作，0表示进行读操作
MemToReg	WB	WB	写回控制，为1时将数据存储器读出数据写回寄存器组，否则将ALU运算结果AluResult写回寄存器组

二、宏观视角

把原来“一条指令一口气做完”的单周期通路，**剪成五段，中间插上寄存器**，让五条指令像流水线上的产品一样错位接力

四大要素

- **五段彩色区域(IF/ID/EX/MEM/WB)**:按功能划分的五个工位，取指→译码→执行→访存→写回，顺序固定

- **四根红色竖条(流水寄存器)**：工位之间的“传送带闸口”，每个时钟脉冲 CLK 来临时，把上一段算完的所有东西一次性锁存，送给下一段，是流水线的物理分界
- **单向主流 + 少数回头路**：数据总体从左往右流；但写回结果要送回最左边的寄存器堆，分支判断结果要送回最左边改 PC——这两条“回头路”是效率瓶颈，也是后续“冒险”问题的根源
- **专用通路，没有总线**：所有连线都是点对点直连，数据各走各的路、互不抢道，所以不需要复杂的总线仲裁，控制也更简单(这正是图中强调单周期专用通路结构的原因)

没画出来的东西

图中没有前递单元、没有冒险检测，是一台“理想流水线”教学模型，真实芯片还需额外处理数据相关

三、微观视角

以**ADD**指令为例

第 1 站：取指 IF

- PC 给出地址，指令存储器按地址取出指令
- 旁边的小加法器顺手算好 PC+4,作为下一条指令的地址
- 末端的多路器听 BranchTaken 指挥：分支成立就改走 BranchAddr,否则照常 PC+4 **过第一道闸：IF/ID 寄存器**，把整条指令和 PC+4 锁住

第 2 站：译码 ID

- 指令头部的 OP 字段送进**控制器**，一次性译出所有控制信号(RegWrite、AluOp、MemWrite 等)
- 指令里的 rs、rt 两个寄存器编号送进**寄存器堆**，同时读出两个操作数 R1、R2
- **符号扩展**单元把 16 位立即数补成 32 位
- RegDst 多路器决定目的寄存器到底写 rt 还是 rd(区分 R 型和 I 型指令) **关键技巧**：控制器只在这一段出现一次，译出的控制信号不马上用完，而是**跟着指令一起“搭车”过闸**，每段要用哪个信号就取哪个，省掉了每段重复设计控制器的成本 **过第二道闸：ID/EX 寄存器**，锁住 R1、R2、立即数、WriteReg# 和一整串控制信号

第 3 站：执行 EX

- 两个多路器(SrcA、SrcB)按 AluSrc 指令，从“寄存器值”和“立即数”里选好两个操作数，喂给 **ALU**
- ALU 算出结果 AluResult:可能是加法结果，也可能是访存地址
- 附带干两件事：
  - 立即数左移 2 位后与 PC+4 相加，算出**分支目标地址** BranchAddr
  - 相等比较器输出 Equal,与 Branch 信号合成 **BranchTaken**,立刻回传 IF 段改 PC,让 beq 只损失一个周期 **过第三道闸：EX/MEM 寄存器**，锁住 ALU 结果、待写数据 RT 和访存控制信号

第 4 站：访存 MEM

- 只有 load/store 指令真正干活：ALU 结果当地址 A,MemWrite 允许时把 RT 从 WD 口写进**数据存储器**，否则从 RD 口读出数据

- 其他指令在这一站只是“路过” **过第四道闸：MEM/WB 寄存器**，同时锁住 ALU 结果和存储器读出的数据

第 5 站：写回 WB

- MemToReg 多路器二选一：运算指令选 ALU 结果，load 指令选存储器读出的数据
- 选出的 WriteData 连同 WriteReg#,**一路向左横跨四级**，写回最开头的寄存器堆，完成闭环 **一个隐藏时序约定**：寄存器堆必须“前半周期写、后半周期读”——同一周期里上一条指令在写、下一条指令在读，靠时钟相位错开避免冲突，图中为简洁没有画出

四、举例图示

